

常见 Linux 命令速查表

一、日常基础命令

1. 查 IP

- 命令: `ifconfig` 或 `ip addr`
- 作用: 查看网络接口信息, 包括 IP 地址。
- 记忆点: `ifconfig` 是 “interface config” 的缩写, `ip addr` 是新版工具, 像查 “网络身份证”。
- 应用举例:
 - `ifconfig` 输出 `eth0: inet 192.168.1.10`, 确认测试服务器 IP。
 - `ip addr show` 检查所有网络接口状态。

2. 查看当前位置

- 命令: `pwd`
- 作用: 显示当前目录的绝对路径。
- 记忆点: “print working directory”, 像问 “我的位置”。
- 应用举例:
 - `pwd` 输出 `/var/log`, 确认日志存放路径。

3. 清屏

- 命令: `clear` 或者使用快捷键 `ctrl + l`
- 作用: 清除终端屏幕内容。
- 记忆点: 像 “清理画布”, 让屏幕整洁。
- 应用举例:
 - 测试时屏幕杂乱, 输入 `clear` 刷新界面。

4. 文件路径表示

- 命令: 无具体命令, 涉及路径规则
- 作用: 表示文件或目录位置, 分为绝对路径 (从 `/` 开始) 和相对路径 (基于当前位置)。
- 记忆点:
 - `/` 是根, 像树的起点。
 - `.` 是当前, `..` 是上一级, 像 “点一下这儿, 点两下往上”。
 - `~` 表示当前用户的主目录/家目录。
- 应用举例:

- 绝对路径: `/etc/nginx/nginx.conf` 指定配置文件。
- 相对路径: `./test.sh` 表示当前目录下的脚本。

5. 切换目录

- 命令: `cd`
- 作用: 更改当前工作目录。
- 记忆点: “change directory”, 像换工作地点。
- 应用举例:
 - `cd /home/user` 进入用户目录检查测试文件。
 - `cd ..` 返回上一级目录。

6. 查找文件位置

- 命令: `find`
- 作用: 在指定路径下查找文件或目录。
- 记忆点: “find” 就是找东西, 加 `-name` 指定名字。
- 应用举例:
 - `find /var -name "error.log"` 定位错误日志。
 - `find . -name "*.py"` 查找当前目录下所有 Python 文件。

7. 查看目录下有哪些文件、文件夹

- 命令: `ls` (类比 Windows 的 `dir`)
- 作用: 列出目录内容。
- 记忆点: “list” 的缩写, `-l` 详细, `-a` 包括隐藏。
- 应用举例:
 - `ls -l /tmp` 查看临时目录下文件详情。
 - `ls -a` 显示隐藏文件如 `.gitignore`。

8. 文件夹创建

- 命令: `mkdir`
- 作用: 创建新目录。
- 记忆点: “make directory”, 像建新文件夹。
- 应用举例:
 - `mkdir test_cases` 创建测试用例目录。
 - `mkdir -p logs/2025` 递归创建多级目录。

9. 文件创建

- 命令: `touch`
- 作用: 创建空文件或更新文件时间戳。
- 记忆点: “touch” 像轻点一下生成文件。
- 应用举例:
 - `touch test_data.csv` 创建空的测试数据文件。
 - `touch readme.md` 初始化说明文档。

10. 修改文件名

- 命令: `mv`
- 作用: 通过移动到新名称实现重命名。
- 记忆点: “move” 到新名字, 像改名不挪窝。
- 应用举例:
 - `mv old_test.txt new_test.txt` 重命名文件。
 - `mv draft final` 将目录从 `draft` 改为 `final`。

11. 删除文件/目录

- 命令: `rm`
- 作用: 删除文件或目录。
- 记忆点: “remove”, `-r` 递归删除目录, `-f` 强制执行。
- 应用举例:
 - `rm temp.txt` 删除临时文件。
 - `rm -rf old_logs` 删除整个旧日志目录 (谨慎使用!)。

12. 文件移动

- 命令: `mv`
- 作用: 将文件或目录移动到新位置。
- 记忆点: “move”, 像搬家到新地址。
- 应用举例:
 - `mv report.pdf /home/user/docs` 移动报告到文档目录。
 - `mv *.log logs/` 将所有日志文件移到 `logs` 文件夹。

13. 文件复制

- 命令: `cp`
- 作用: 复制文件或目录。
- 记忆点: “copy” , `-r` 用于递归复制目录。
- 应用举例:
 - `cp config.ini config_backup.ini` 备份配置文件。
 - `cp -r src/ backup/` 复制整个 `src` 目录到 `backup` 。

14. 文件内容查看

- 命令: `cat` 、 `less` 、 `more`
- 作用: 查看文件内容。
- 记忆点:
 - `cat` 是 “concatenate” , 一次性输出。
 - `less` 和 `more` 分页看, 像翻书, 可以通过如下按键实现翻页。
 - `b` : 上一页 (backward)
 - `f` : 下一页 (forward)
 - `q` : 退出 (quit)
 - `/` : 后面可以按照指定关键词查看
- 应用举例:
 - `cat /var/log/syslog` 查看系统日志。
 - `less large_file.log` 分页浏览大文件, 按 `q` 退出。

15. 内容编辑

- 命令: `vi` 或 `vim`
- 作用: 编辑文件内容。
- 记忆点:
 - `vim` 按 `i` 进入编辑, `:wq` 保存退出, `:q!` 不保存强制退出。
- 应用举例:
 - `vim test_script.sh` 修改测试脚本内容。

16. 内容搜索

二、其他命令补充

1. zip 压缩

- 命令: `zip`
- 作用: 将文件或目录压缩为 `.zip` 格式。
- 记忆点: “zip” 就是拉链, 把文件打包。
- 应用举例: `zip -r test.zip test_dir/` 将 `test_dir` 压缩为 `test.zip`。
- 应用场景: 测试完成后, 压缩日志文件 (如 `logs/`) 便于上传或归档。

2. unzip 解压缩

- 命令: `unzip`
- 作用: 解压 `.zip` 文件。
- 记忆点: “unzip” 解开拉链, 释放内容。
- 应用举例: `unzip test.zip` 解压 `test.zip` 到当前目录。
- 应用场景: 从开发团队收到 `.zip` 格式的测试包, 解压后部署测试。

3. tar 压缩和解压缩

- 命令: `tar`
- 作用: 打包文件 (不压缩) 或结合 `gzip` 压缩为 `.tar.gz`。
- 记忆点: `tar` 是 “tape archive”, 像磁带打包, `-c` 创建, `-x` 提取, `-z` 加压缩。
- 应用举例:
 - `tar -czf logs.tar.gz logs/` 压缩 `logs` 为 `logs.tar.gz`。
 - `tar -xzf logs.tar.gz` 解压 `logs.tar.gz`。
- 应用场景: 测试环境迁移时, 打包配置文件和数据 (如 `/etc/nginx`), 传输后解压还原。

4. grep 搜索

- 命令: `grep`
- 作用: 在文件中搜索匹配指定模式的文本行。
- 记忆点: “global regular expression print” 的缩写, 像 “全局搜索打印”, 擅长找关键词。
 - 常用选项: `-i` (忽略大小写)、`-r` (递归搜索)、`-n` (显示行号)。
- 应用举例:
 - `grep "ERROR" error.log` 查找 `error.log` 中含 “ERROR” 的行。
 - `ps aux | grep "nginx"` 从进程列表中筛选含 “nginx” 的行。
- 应用场景:
 - 日志分析: 测试时快速定位错误日志中的关键词 (如 “FAIL” 或 “Exception”)。
 - 管道结合: 与其他命令 (如 `ps`、`cat`) 配合, 过滤关键信息。

5. 管道符

- 命令: `|`
- 作用: 将前一个命令的输出作为后一个命令的输入。
- 记忆点: 像水管, 把结果“流”到下一个命令。
- 应用举例: `ls -l | grep "test"` 筛选含“test”的文件。
- 应用场景: 查看日志时, `cat error.log | grep "ERROR"` 过滤错误信息。

6. 重定向

- 命令: `>` (覆盖)、`>>` (追加)
- 作用: 将命令输出重定向到文件。
- 记忆点: `>` 是倒水覆盖, `>>` 是加水追加。
- 应用举例:
 - `echo "start" > status.txt` 写入“start”到文件。
 - `ls -l >> dir_list.txt` 追加目录列表。
- 应用场景: 测试脚本运行时, 将结果输出到文件保存, 如 `python test.py > result.txt`。

7. 权限查看

- 命令: `ls -l` 或者 `ll`
- 作用: 查看文件或目录的权限、所有者等信息。
- 记忆点: `-l` 是“long list”, 显示详细权限如 `rwxr-xr-x`。
- 应用举例: `ls -l /home/user/test.sh` 查看脚本权限。
- 应用场景: 排查脚本无法执行时, 检查是否缺少执行权限 (`x`)。

8. 权限修改

- 命令: `chmod`
- 作用: 修改文件或目录权限。
- 记忆点: “change mode”, 数字法 (如 `755`) 或字母法 (如 `u+x`)。
- 应用举例:
 - `chmod 755 test.sh` 设置脚本可执行。
 - `chmod u+w file.txt` 给用户添加写权限。
- 应用场景: 测试脚本部署后, 赋予执行权限以运行自动化测试。

9. 临时提权

- 命令: `sudo`
- 作用: 以超级用户权限执行命令。
- 记忆点: “superuser do” , 临时当 “老板” 。
- 应用举例: `sudo systemctl restart nginx` 重启服务。
- 应用场景: 测试需要修改系统文件 (如 `/etc/hosts`) 时, 提升权限操作。

10. 查看进程

- 命令: `ps`
- 作用: 显示当前运行的进程。
- 记忆点: “process status” , `aux` 显示所有用户进程。
- 应用举例: `ps -aux | grep "python"` 查看 Python 进程。
- 应用场景: 检查测试脚本是否在后台运行, 避免重复启动。

11. 杀进程

- 命令: `kill`
- 作用: 终止指定进程。
- 记忆点: `kill` 加 PID (进程 ID) , `-9` 强制杀死。
- 应用举例: `kill -9 1234` 强制终止 PID 为 1234 的进程。
- 应用场景: 测试程序卡死时, 找到 PID 后杀掉, 释放资源。

12. 查端口

- 命令: `netstat` 或 `ss`
- 作用: 查看网络端口使用情况。
- 记忆点: `netstat -anptu` 显示监听端口, `ss` 是新版替代。
- 应用举例:
 - `netstat -anptu | grep 8080` 检查 8080 端口占用。
 - `ss -tuln` 列出所有 TCP/UDP 端口。
- 应用场景: 测试服务启动失败, 检查端口是否被占用。

13. 查日志 (head/tail)

- 命令: `head` 、 `tail`
- 作用: 查看文件开头或结尾内容。
- 记忆点: `head` 是头, `tail` 是尾, `-n` 指定行数。
- 应用举例:

- `head -n 10 error.log` 查看日志前 10 行。
- `tail -n 20 access.log` 查看日志最后 20 行。
- 应用场景: 排查问题时, `tail -f error.log` 实时监控日志输出。

14. 查磁盘空间

- 命令: `df`
- 作用: 显示磁盘使用情况。
- 记忆点: “disk free”, `-h` 以人类可读格式显示。
- 应用举例: `df -h` 查看磁盘剩余空间。
- 应用场景: 测试数据积累导致磁盘满, 检查并清理无用文件。

15. yum 软件安装

- 命令: `yum`
- 作用: 在 CentOS/RHEL 系统上安装、管理软件包。
- 记忆点: “yellowdog updater modified”, 像超市买软件。
- 应用举例:
 - `yum install -y nginx` 安装 Nginx。
 - `yum update` 更新所有软件包。
- 应用场景: 测试环境缺少工具 (如 `vim`), 用 `yum` 快速安装。

16. systemctl 服务管理

- 命令: `systemctl`
- 作用: 管理系统服务 (如启动、停止、查看状态)。
- 记忆点: “system control”, 常用 `start`、`stop`、`status`。
- 应用举例:
 - `systemctl start docker` 启动 Docker 服务。
 - `systemctl status nginx` 查看 Nginx 运行状态。
- 应用场景: 测试服务部署后, 检查服务是否正常运行或重启服务。

三、docker相关命令扩展

3.1 Docker 相关命令

1. 查看 Docker 版本

- 命令: `docker --version` 或 `docker version`
- 作用: 显示 Docker 的版本信息。
- 记忆点: `--version` 是简查, `version` 更详细, 像问 “Docker 你几岁”。
- 应用举例:
 - `docker --version` 输出 `Docker version 20.10.12`。
 - `docker version` 显示客户端和服务端信息。
- 应用场景: 确认测试环境的 Docker 版本是否符合项目要求。

2. 查看运行中的容器

- 命令: `docker ps`
- 作用: 列出当前运行的容器。
- 记忆点: “process status” 的缩写, `-a` 显示所有容器 (包括停止的)。
- 应用举例:
 - `docker ps` 查看运行中的容器列表。
 - `docker ps -a` 显示所有容器, 包括已停止的。
- 应用场景: 检查测试服务 (如 Web 应用容器) 是否正常运行。

3. 启动/停止/重启容器

- 命令: `docker start` / `docker stop` / `docker restart`
- 作用: 管理容器运行状态。
- 记忆点: 像开关按钮, 后面跟容器 ID 或名称。
- 应用举例:
 - `docker start myapp` 启动名为 `myapp` 的容器。
 - `docker stop myapp` 停止它。
 - `docker restart myapp` 重启容器。
- 应用场景: 测试中调整容器状态, 如重启数据库容器以应用新配置。

4. 运行容器

- 命令: `docker run`
- 作用: 从镜像创建并运行一个新容器。
- 记忆点: “run” 是启动, `-d` 后台运行, `--name` 命名。
- 应用举例:
 - `docker run -d --name web nginx` 后台运行一个 Nginx 容器。
 - `docker run -it ubuntu bash` 启动 Ubuntu 容器并进入交互模式。

- **应用场景:** 快速部署测试环境，如运行一个临时的 Redis 容器测试缓存功能。

5. 查看容器日志

- **命令:** `docker logs`
- **作用:** 查看容器输出日志。
- **记忆点:** “logs” 像日记本，`-f` 实时跟踪。
- **应用举例:**
 - `docker logs web` 查看 `web` 容器的日志。
 - `docker logs -f myapp` 实时监控日志。
- **应用场景:** 排查测试服务异常，查看容器日志中的错误信息。

6. 进入容器内部

- **命令:** `docker exec`
- **作用:** 在运行中的容器内执行命令或进入 shell。
- **记忆点:** “execute” 在容器里做事，`-it` 交互模式。
- **应用举例:**
 - `docker exec -it myapp bash` 进入 `myapp` 容器的 `bash`。
 - `docker exec myapp ls` 执行容器内的命令。
- **应用场景:** 测试时进入容器修改配置文件或检查环境变量。

7. 删除容器

- **命令:** `docker rm`
- **作用:** 删除指定容器。
- **记忆点:** “remove”，`-f` 强制删除运行中的容器。
- **应用举例:**
 - `docker rm myapp` 删除停止的 `myapp` 容器。
 - `docker rm -f myapp` 强制删除运行中的容器。
- **应用场景:** 测试完成后清理无用容器，释放资源。

8. 查看镜像

- **命令:** `docker images`
- **作用:** 列出本地 Docker 镜像。
- **记忆点:** “images” 是镜像清单，像查看库存。
- **应用举例:** `docker images` 显示镜像名称、标签、大小等。
- **应用场景:** 确认测试所需的镜像（如 `mysql:5.7`）是否已拉取。

9. 拉取镜像

- 命令: `docker pull`
- 作用: 从 Docker Hub 下载镜像。
- 记忆点: “pull” 是拉过来, 像取快递。
- 应用举例: `docker pull redis:latest` 下载最新 Redis 镜像。
- 应用场景: 测试前拉取项目依赖的镜像 (如 Nginx、Python) 。

10. 删除镜像

- 命令: `docker rmi`
- 作用: 删除本地镜像。
- 记忆点: “remove image”, 注意容器依赖时需先删除容器。
- 应用举例: `docker rmi redis:latest` 删除 Redis 镜像。
- 应用场景: 测试版本升级后, 清理旧镜像以节省磁盘空间。

3.2 Docker Compose 相关命令

1. 启动服务

- 命令: `docker-compose up`
- 作用: 根据 `docker-compose.yml` 文件启动所有服务。
- 记忆点: “up” 是启动, `-d` 后台运行。
- 应用举例:
 - `docker-compose up` 前台启动服务。
 - `docker-compose up -d` 后台启动。
- 应用场景: 测试微服务架构时, 一键启动 Web、数据库等多个容器。

2. 停止服务

- 命令: `docker-compose down`
- 作用: 停止并删除 `docker-compose` 创建的容器、网络等。
- 记忆点: “down” 是关闭, 清理干净。
- 应用举例: `docker-compose down` 停止并移除所有服务。
- 应用场景: 测试结束后清理环境, 避免资源占用。

3. 查看服务状态

- 命令: `docker-compose ps`
- 作用: 显示 `docker-compose` 管理的容器状态。
- 记忆点: 与 `docker ps` 类似, 聚焦当前项目。

- **应用举例:** `docker-compose ps` 查看服务运行情况。
- **应用场景:** 确认测试环境中的服务（如 API 和 DB）是否正常运行。

4. 查看服务日志

- **命令:** `docker-compose logs`
- **作用:** 查看所有服务的日志。
- **记忆点:** “logs” 是日记, `-f` 实时查看。
- **应用举例:**
 - `docker-compose logs` 查看所有服务日志。
 - `docker-compose logs -f web` 实时查看 `web` 服务日志。
- **应用场景:** 测试多服务系统时, 排查某个服务（如前端）的异常。

5. 构建服务

- **命令:** `docker-compose build`
- **作用:** 根据 `docker-compose.yml` 中的 `build` 配置构建镜像。
- **记忆点:** “build” 是建造, 生成自定义镜像。
- **应用举例:** `docker-compose build` 构建项目中的服务镜像。
- **应用场景:** 测试本地开发的 Docker 镜像, 确保代码更改生效。

6. 重启服务

- **命令:** `docker-compose restart`
 - **作用:** 重启指定或所有服务。
 - **记忆点:** “restart” 像重启按钮。
 - **应用举例:**
 - `docker-compose restart` 重启所有服务。
 - `docker-compose restart web` 重启 `web` 服务。
 - **应用场景:** 测试配置更新后, 重启服务验证效果。
-